

Introdução a Algoritmos

Aula 3 – Decisões

Raoni F. S. Teixeira · 2s/2026

Objetivos desta aula

Ao final desta aula, você deverá ser capaz de:

1. Escrever condicionais `if`, `if/else` e `if-else-if` para implementar decisões com dois ou mais casos mutuamente exclusivos;
2. Identificar o bug do `dangling else` e corrigi-lo com o uso de chaves;
3. Distinguir o operador de atribuição `=` do operador de igualdade `==` e explicar por que a confusão não gera erro de compilação;
4. Ordenar condições num `if-else-if` de forma que a lógica produza o resultado correto para todas as entradas válidas.

1 A peça “decisão”: escolher um caminho

Nas duas primeiras aulas, nossos programas executavam todas as instruções sempre, na mesma ordem. Mas a maioria dos problemas reais exige que o programa **tome decisões**: calcular desconto apenas se o valor ultrapassar um limite, exibir mensagem de erro apenas se a entrada for inválida, seguir um caminho diferente dependendo do valor lido.

Essa capacidade — executar instruções seletivamente, dependendo de uma condição — é a peça “decisão” do nosso quebra-cabeça. Em C, ela é implementada pelo comando `if`.

Para que o `if` funcione, precisamos antes entender como expressar condições. Isso nos leva ao conceito de **valor booleano** e às **expressões relacionais**.

2 Valores booleanos

Definição — Valor Booleano

Um valor booleano representa o resultado de uma afirmação lógica: ele pode ser **verdadeiro** ou **falso** — e nada mais. O nome homenageia George Boole (1815–1864), matemático britânico que formalizou a álgebra da lógica.

Exemplos de afirmações e seus valores booleanos:

- “O céu é azul.” → **verdadeiro**
- “ $2 + 2 = 5$.” → **falso**
- “O número 7 é par.” → **falso**
- “A variável `x` contém um valor maior que zero.” → **depende do valor de `x`**

O último exemplo é fundamental: em programação, as condições geralmente envolvem variáveis cujo valor só é conhecido em tempo de execução. O programa precisa avaliá-las **enquanto roda**, e não no momento em que foi escrito.

2.1 Booleanos em C

C não tem um tipo booleano nativo (ele foi adicionado apenas no padrão C99, com `<stdbool.h>`). Por convenção histórica, C usa inteiros:

- **0** representa falso;
- **qualquer valor diferente de zero** representa verdadeiro.

Isso significa que a expressão `if (5)` é equivalente a `if (verdadeiro)`, e o bloco sempre será executado. Esse comportamento é usado em alguns idiomas avançados de C, mas deve ser evitado por iniciantes.

3 Expressões relacionais

Uma expressão relacional **compara** dois valores e produz um resultado booleano (0 ou 1 em C). Os operadores relacionais de C são:

Operador	Significado	Verdadeiro quando...
<code>==</code>	Igualdade	a e b têm o mesmo valor
<code>!=</code>	Diferença	a e b têm valores distintos
<code>></code>	Maior que	a é estritamente maior que b
<code><</code>	Menor que	a é estritamente menor que b
<code>>=</code>	Maior ou igual	a é maior que ou igual a b
<code><=</code>	Menor ou igual	a é menor que ou igual a b

Atenção

A confusão mais comum em C: `=` é atribuição; `==` é comparação. Escrever `if (x = 5)` em vez de `if (x == 5)` **não gera erro de compilação** — o compilador interpreta como “atribua 5 a x e teste se o resultado é verdadeiro” (que sempre será, pois `5 != 0`). O programa compila e roda, mas se comporta de forma errada. Muitos compiladores emitem um **aviso** para isso — sempre leia os avisos.

4 O comando if

O comando `if` executa um bloco de instruções **somente se** uma condição for verdadeira.

```
if (condicao) {  
    /* executado apenas se condicao for verdadeira */  
    instrucao1;  
    instrucao2;  
}
```

Se o bloco contiver apenas uma instrução, as chaves são tecnicamente opcionais. No entanto, omiti-las é uma fonte frequente de bugs — a prática recomendada é **sempre usar chaves**.

Exemplo — Verificar se um número é ímpar

```
int n;
scanf("%d", &n);
if (n % 2 != 0) {
    printf("O número é ímpar.\n");
}
```

O operador % calcula o resto da divisão. Se o resto da divisão por 2 for diferente de zero, o número é ímpar.

5 O comando if/else

Muitas vezes queremos executar ações **diferentes** dependendo da condição — não apenas agir quando ela é verdadeira, mas também quando é falsa. Para isso, usamos if/else:

```
if (condicao) {
    /* executado se verdadeiro */
} else {
    /* executado se falso */
}
```

Exatamente um dos dois blocos será executado — nunca ambos, nunca nenhum.

Exemplo — Classificar número como positivo ou não-positivo

```
int n;
scanf("%d", &n);
if (n > 0) {
    printf("Número positivo.\n");
} else {
    printf("Número negativo ou zero.\n");
}
```

5.1 O perigo do else sem chaves: o dangling else

Considere o código abaixo — aparentemente correto, mas com um bug sutil:

```
if (cond1)
    if (cond2)
        printf("A\n");
else
    printf("B\n"); /* a qual if esse else pertence? */
```

Pela indentação, parece que o else pertence ao if (cond1). Mas em C, o else **sempre se associa ao if mais próximo que ainda não tem um else**. Portanto, o código acima é equivalente a:

```
if (cond1) {
    if (cond2)
        printf("A\n");
    else
```

```
        printf("B\n");    /* pertence ao if(cond2), não ao if(cond1) */
    }
```

Esse problema — chamado de **dangling else** — é uma das armadilhas mais antigas de C. A solução é sempre usar chaves:

```
if (cond1) {
    if (cond2) {
        printf("A\n");
    }
} else {
    printf("B\n");    /* agora inequivocamente do if(cond1) */
}
```

Regra prática: use chaves em todos os blocos de `if` e `else`, mesmo quando há apenas uma instrução. O custo é zero; o benefício é eliminar uma classe inteira de bugs.

6 Estrutura if-else-if

Quando há três ou mais casos mutuamente exclusivos, encadeamos `if-else-if`:

```
if (condicao1) {
    /* caso 1 */
} else if (condicao2) {
    /* caso 2 */
} else if (condicao3) {
    /* caso 3 */
} else {
    /* nenhum dos casos anteriores */
}
```

A avaliação ocorre de cima para baixo: assim que uma condição for verdadeira, seu bloco é executado e os demais são ignorados. O bloco `else` final (sem condição) serve como caso padrão — ele captura qualquer situação não tratada pelas condições anteriores.

Exemplo — Classificação de notas

```
float nota;
scanf("%f", &nota);

if (nota >= 9.0) {
    printf("Conceito A\n");
} else if (nota >= 7.0) {
    printf("Conceito B\n");
} else if (nota >= 5.0) {
    printf("Conceito C\n");
} else {
    printf("Reprovado\n");
}
```

Observe que as condições são ordenadas da maior para a menor. Se estivessem na ordem inversa (começando por `nota >= 5.0`), uma nota 9.5 seria classificada como conceito C — o primeiro `if` que a capturasse, incorretamente.

A ordem das condições no `if-else-if` importa.

6.1 Comparação: ifs independentes vs. if-else-if

Há uma diferença importante entre uma sequência de ifs independentes e uma cadeia if-else-if:

```
/* Versão A: ifs independentes – todos os testes são avaliados */
if (x > 0) printf("positivo\n");
if (x > 10) printf("maior que 10\n");
if (x > 100) printf("maior que 100\n");

/* Versão B: if-else-if – para no primeiro verdadeiro */
if (x > 100) {
    printf("maior que 100\n");
} else if (x > 10) {
    printf("entre 11 e 100\n");
} else if (x > 0) {
    printf("entre 1 e 10\n");
}
```

Use ifs independentes quando as condições **não são mutuamente exclusivas** (um valor pode satisfazer mais de uma). Use if-else-if quando os casos são **mutuamente exclusivos** — só um deles deve ser tratado.

7 Exercícios

1. Escreva um programa que leia um inteiro e imprima:
 - “SIM” se ele for par e maior que 10, **ou** se for ímpar e menor que 50;
 - “NAO” em qualquer outro caso.

(Dica: tente primeiro com ifs aninhados; depois simplifique usando operadores lógicos — que veremos em detalhe na próxima aula.)

2. Escreva um programa que leia três números inteiros e os imprima em ordem crescente. *(Sem usar laços — apenas ifs e variáveis auxiliares.)*
3. Analise o código abaixo. Para quais valores de n ele produz um resultado incorreto? Corrija-o.

```
int n;
scanf("%d", &n);
if (n > 0)
    printf("positivo\n");
if (n == 0)
    printf("zero\n");
else
    printf("negativo\n");
```